

02 - Software Requirement Specification (SRS)

1. Введение

1.1 Цель документа

Данный документ (Software Requirements Specification, SRS) имеет своей целью определить все требования к игровому программному продукту “Neon Runner”. В рамках SRS задаётся основа для понимания функционала, архитектуры, технических и качественных параметров, а также ограничений и предположений о среде исполнения.

Целью данного документа является:

- Определить функциональные требования к игровым механикам, взаимодействию с пользователем, прогрессу и достижению конечной цели в игре.
- Описать нефункциональные требования, включающие производительность, надёжность, удобство использования, совместимость и расширяемость.
- Установить чёткие критерии приёмки, которые позволят определить, соответствует ли готовый программный продукт ожиданиям и спецификациям, изложенным в данном документе.
- Быть руководством для всех участников процесса разработки — гейм-дизайнеров, программистов, тестировщиков и других заинтересованных сторон, чтобы обеспечить единое понимание целей, механик и ограничений проекта.

Данный документ охватывает все ключевые аспекты разработки игры, включая описание игрового процесса, взаимодействие с окружением и врагами, управление ресурсами (аптечки, оружие, механика замедления времени, etc.), а также технические и эксплуатационные условия, в которых игра будет функционировать.

1.2 Область применения проекта

Игра “Neon Runner” — новый 2D аркадный киберпанк-платформер с вертикальным скроллингом и элементами выживания. Главный герой — репликант, стремящийся преодолеть ограничения своей искусственной жизни, поднимаясь по этажам корпорации Wallace.

Ключевые особенности:

- **Жанр.** Вертикальный экшен-платформер с акцентом на ловкость, точные прыжки и своевременные действия.
- **Сеттинг.** Киберпанк-антураж, неоновое освещение, индустриальные ландшафты, корпоративная архитектура.
- **Целевая аудитория.** Игроки, ценящие динамичный геймплей, смесь платформенных и аркадных механик, а также атмосферу киберпанка. Проект ориентирован на любителей сложных платформеров со справедливым балансом между показателями игрока и противостоящим ему окружением, где важна точность и тактика, а также на поклонников эстетики “Blade Runner” и футуристичных миров.

Данный документ SRS считается актуальным для всех этапов разработки: от первоначального прототипирования до финального тестирования и окончательной приёмки продукта.

1.3 Термины, определения и сокращения

Для обеспечения единообразия понимания требований приводятся основные термины, используемые в проекте:

- **Репликант.** Искусственно созданный биологический организм, внешне неотличимый от человека, но имеющий встроенные генетические ограничения по сроку жизни. Игрок управляет репликантом.
- **Бегущий по лезвию.** В данном контексте — агент, специализирующийся на обнаружении и уничтожении “нестабильных” репликантов. В игре такие агенты встречаются в качестве мини-боссов.
- **Аптечка.** Специальный ресурс, продлевающий время жизни репликанта. Без регулярного сбора аптек оставшееся время жизни стремительно убывает.

- **Неоновый элемент (неон).** Визуальный и стилистический элемент, символизирующий футуристический киберпанковый антураж. Неоновое освещение, контрастные оттенки и яркие акценты подчеркивающие технологичность и искусственность окружающей среды.
- **Замедление времени.** Игровая механика, позволяющая игроку временно снизить скорость течения игрового времени, что даёт возможность точнее планировать прыжки, обходить врагов или избегать снарядов.
- **Корпорация Wallace.** Организация, контролирующая производство и обслуживание репликантов, а также установившая ограничения на срок их жизни.

При необходимости в документе могут использоваться сокращения. Все сокращения, специфичные для проекта, будут определены при первом их упоминании.

1.4 Ссылки на используемые документы и стандарты

Для подготовки и реализации данного проекта используются следующие стандарты, руководства и ресурсы:

- **Стандарты и спецификации:**
 - ISO/IEC 9126 или ISO/IEC 25010 (качество ПО)
 - IEEE 830 (рекомендуемые практики для SRS)
- **Техническая документация и библиотеки:**
 - Документация по Pygame (официальный сайт и репозиторий) ((вставить ссылки))
 - Руководства по использованию Python 3.x и сторонних библиотек, необходимых для отрисовки, звука, ввода с устройств управления ((вставить ссылки))
- **Гейм-дизайн документы и прототипы:**
 - Предварительный GDD (Game Design Document), на основе которого формируется данное SRS ((вставить ссылку))
 - Внутренние спецификации спрайтов, анимаций, звуковых эффектов, музыкальных треков и прочих ассетов ((вставить ссылки))
 - Прототипы уровней, интерфейса, дизайн-макеты экранов меню, HUD и конечной сцены ((вставить ссылки на репозиторий и документацию к ним, если есть))
- **Интерфейсные руководства (если применимо):**
 - Внутренние гайды по UI/UX, используемые в компании-разработчике
 - Рекомендации по визуальному стилю, оформлению неоновой эстетики, читаемости текстов на различных разрешениях экрана

2. Общий обзор системы

2.1 Общая функциональность

В основе игры “Neon Runner” лежит вертикальный акцент на геймплей, где игрок управляет репликантом в киберпанковой среде, преодолевая этажи высотного здания корпорации Wallace. Ключевые элементы функциональности включают:

- **Вертикальное продвижение персонажа.** Игрок совершает точные и своевременные прыжки по платформам, уступам и техническим конструкциям, встречающимся при подъёме вверх. Движение не ограничивается классическим горизонтальным перемещением — основной вектор прогресса направлен строго вверх, отражая идею “преодоления вертикального барьера”.
- **Сбор аптечек для продления жизни.** Персонаж стартует с ограниченным таймером, который неумолимо уменьшает оставшиеся секунды существования. Аптечки, спрятанные в труднодоступных нишах или мини-лабораториях внутри здания, продлевают время жизни репликанта. Таким образом, поиск и эффективный сбор аптечек — фундаментальная составляющая успешного прохождения игры.

- **Сражения и уклонение от врагов.** На пути к генетической модификации репликанту противостоят различные охранные системы: дроны, патрульные киборги и агенты типа “бегущие по лезвию”. Игроку необходимо быстро принимать решения — вступать в бой или обходить противника, используя элементы платформинга, стелса и доступного вооружения. В этом аспекте отражается боевая функциональность — атаки в ближнем бою с помощью электро-дубинки или дальние выстрелы из импульсного пистолета.
- **Использование механики замедления времени.** Игрок может замедлить ход времени, чтобы совершить более точные и выверенные прыжки, уклониться от опасных атак или отрегулировать тайминг для успешного парирования вражеских выстрелов. Эта функция добавляет глубину геймплею, позволяя чувствовать себя главным в ситуации, в условиях, требующих максимально точной реакции.
- **Достижение финальной цели.** Главное предназначение всех перечисленных функций — дойти до верхних уровней башни, победить финального противника и получить генетическую модификацию, которая отменяет ограниченный срок жизни репликанта. В этот момент главная механика сборки ресурсов (аптечек) и борьбы с врагами переходит в кульминационную точку, завершаясь повествовательным элементом в сюжете игры и общим удовлетворением игрока от прохождения.

Таким образом требуется, чтобы игрок балансировал между скоростью подъёма, безопасностью, временем жизни и борьбой с врагами, создавая для пользователя уникальный игровой опыт.

2.2 Краткий обзор пользовательских сценариев

Пользовательские сценарии в “Neon Runner” определяют последовательность действий, которые игрок может совершать, а также реагирование игровой системы на эти действия. Они служат логической основой для проектирования внутриигровых механик, игровых объектов и взаимодействий между ними. Рассмотрим ключевые сценарии:

1. Прохождение уровня (от входа в здание до перехода на верхний этаж):

- **Начальное состояние.** Игрок появляется в нижней части вертикальной локации (например, техническом отсеке или шахте). На экране отображается таймер жизни, показатели здоровья и ресурс замедления времени, если он имеется.
- **Основные действия:**
 - Перемещение персонажа: игрок нажимает соответствующие клавиши или кнопки геймпада, чтобы двигаться по горизонтали, прыгать и цепляться за платформы.
 - По мере продвижения вверх сцена плавно скроллируется, генерируя или показывая следующие сегменты уровня (реализуется через механику вертикального скролла или поэтажной подгрузки).
 - Игрок ищет аптечки в труднодоступных нишах и тайниках. При пересечении хитбокса персонажа с хитбоксом аптечки запускается внутриигровое событие: обновляется таймер жизни, проигрывается анимация, исчезает объект аптечки.
 - В процессе движения игрок может остановиться для точного прицеливания, использовать замедление времени для сложных прыжков или активации оружия при критических ситуациях.
- **Условия завершения.** Когда игрок достигает верхнего контрольного сегмента уровня (например, определённая высота или конкретная выходная платформа), происходит переход на следующий этаж или уровень. Данный переход может сопровождаться подсчётом очков, воспроизведением короткой анимации перехода и обновлением состояния персонажа.

2. Взаимодействие игрока с препятствиями, врагами и предметами:

- **Препятствия:**
 - Игрок может встретить движущиеся платформы, разрушаемые поверхности, лазерные заслоны или электрические ловушки.
 - При столкновении или пересечении с препятствием срабатывают определённые события: уменьшение здоровья персонажа, временная блокировка пути, активация мини-головоломок.
 - Программно это реализуется через объекты класса “Obstacle” с определёнными свойствами (координаты, тип взаимодействия, триггеры при контакте).

- **Враги (дроны, патрульные, агенты – “бегущие по лезвию”):**

- Когда игрок входит в зону агро врага (определённый радиус или прямоугольная область), враг начинает действовать враждебно — например, стрелять или двигаться навстречу персонажу.
- При атаке игрока по врагу (ближним или дальним оружием) запускается логика уменьшения очков здоровья врага, проигрывания анимаций попадания. Если здоровье врага падает до нуля, он исчезает (анимация взрыва дрона, падение патрульного, деактивация агента).
- В коде это реализуется через классы врагов с прописанными паттернами поведения (AI), событиями “onHit”, “onDeath” и функциями обнаружения персонажа.

- **Предметы (аптечки, батареи для оружия, усилители замедления времени, ключи или коды доступа):**

- При пересечении персонажа с объектом предмета наступает событие его “поднятия”.
- Каждый предмет имеет прописанные эффекты: аптечка увеличивает таймер жизни, батарея перезаряжает оружие, усилитель пополняет ресурс замедления времени.
- Обработчик события изменяет внутреннее состояние персонажа (обновляет переменные таймера, счётчик патронов) и удаляет предмет из игрового мира.

3. Победа над финальным боссом и получение генетической модификации:

- **Начальное состояние.** Игрок достигает верхнего уровня здания, где находится финальный босс (прототипный ультра-репликант).

- **Процесс сражения:**

- Враг имеет несколько фаз, каждая из которых управляется скриптами AI: изменение паттернов атаки, вызов подкрепления (дронов), увеличенная сложность уклонения.
- Игрок использует свои навыки: точные прыжки, замедление времени, оружие ближнего и дальнего боя, уклонение.
- Каждое попадание по боссу уменьшает его здоровье, отображаемое в UI. По достижении нулевого здоровья босс проигрывает финальную анимацию поражения.

- **Получение модификации:**

- После победы над боссом игровой процесс переходит к сцене в генетической лаборатории. Игрок активирует механизм модификации (например, подходя к интерактивному объекту – капсуле).
- Срабатывают скрипты, обновляющие состояние персонажа: таймер жизни перестаёт уменьшаться, на модель персонажа могут применяться новые визуальные эффекты.
- Игровая логика завершается показом финального ролика (cutscene), финальных титров или статистики прохождения.

Во всех сценариях должны быть чётко определены события, триггеры, условия переходов и взаимодействий. Разработчики должны иметь возможность легко связать логику игры с реализацией классов, функций и обработчиков событий. Также это обеспечит тестировщикам и дизайнерам понятную структуру, на основе которой можно проверять корректность геймплея, баланс и соответствие функциональным требованиям.

2.3 Классы/характеристики пользователей

Игра “Neon Runner” рассчитана на широкий круг игроков, интересующихся аркадными платформерами, киберпанковым антуражем и интенсивным, динамичным геймплеем. Основные характеристики пользовательской аудитории включают:

1. Целевая аудитория:

- **Хардкорные геймеры:** Опытные игроки, ценящие высокую сложность, требовательную реакцию и продуманные геймплейные механики. Такие пользователи готовы преодолевать сложные уровни, учиться на ошибках, повторять попытки до тех пор, пока не достигнут идеального результата. Им особенно важна глубина геймплея, баланс и отсутствие упрощений, снижающих соревновательный интерес.

- **Казуальные игроки:** Игроки, предпочитающие относительно лёгкий вход в игру, возможность постепенного освоения управления, подсказки и визуальные маркеры для понимания ситуации. Хотя они не стремятся к максимальным испытаниям, им важна доступность, стиль и атмосферность проекта. Для таких пользователей будут актуальны опциональные настройки сложности, а также простые, интуитивно понятные элементы геймплея, позволяющие получать удовольствие от игрового процесса, даже если они не стремятся к рекордам или глубокому освоению всех механик.

2. **Платформа.** Игра рассчитана на персональные компьютеры под управлением распространённых операционных систем:

- **Windows:** Основная целевая платформа с широкой пользовательской базой. Разработка с учётом особенностей Windows-окружения, драйверов, типовых разрешений экрана.
- **Linux:** Поддержка для технически подкованной аудитории, предпочитающей открытые платформы. Игра будет доступна для распространённых дистрибутивов, требуя минимальной адаптации и соблюдения системных зависимостей.
- **macOS:** Поддержка пользователей Apple, применение совместимых с macOS билдов и учет специфик рендеринга и ввода.

Указанные адаптации к разным платформам подразумевают тестирование производительности, корректную работу ввода (клавиатура, мышь, геймпад), стабильность и отсутствие критических багов в каждом целевом окружении. Разработчики смогут пользоваться кроссплатформенными возможностями Pygame, чтобы обеспечить приблизительно одинаковый пользовательский опыт на всех поддерживаемых системах.

Учёт интересов и навыков хардкорных и казуальных геймеров, а также кроссплатформенная поддержка должны обеспечить потенциальное расширение потенциальной аудитории игры, обеспечить удобный вход в геймплей и удовлетворение разнообразных предпочтений пользователей.

2.4 Ограничения

При разработке “Neon Runner” учитываются как технические, так и технологические ограничения, влияющие на производительность и совместимость проекта. В данном разделе перечислены основные факторы, которые определяют архитектуру и реализацию игрового кода.

1. Технические ограничения:

- **Производительность на средних ПК:** Игра ориентирована на бесперебойную работу при стандартных для 2D-игр условиях, предполагающих частоту кадров около 60 FPS на системах со средними характеристиками. Под “средним” ПК можно понимать конфигурацию с бюджетным современным процессором, 8–16 ГБ ОЗУ и встроенной или начального уровня дискретной графикой.
- **Требования к CPU:** Обработка логики игры, физики передвижения, коллизий, ИИ врагов и управление ресурсами (аптечками, таймерами, замедлением времени) происходит преимущественно на центральном процессоре. Необходима оптимизация алгоритмов для обеспечения стабильной частоты кадров без видимых задержек и “пропагов”.
- **Требования к GPU:** Хотя Pygame в основном опирается на программный рендеринг через SDL, современное аппаратное окружение почти всегда имеет доступ к графическим ускорителям. Использование GPU не является обязательным, однако игра выиграет от аппаратного ускорения рендеринга, если оно будет доступно (например, через SDL2 и соответствующие модули). При этом целевой рендеринг — 2D-спрайты и эффекты пост-обработки (неоновые свечения, полу-прозрачные слои), которые, в идеале, могут быть ускорены GPU. Если видеоускорение недоступно (например, на некоторых системах Linux без проприетарных драйверов), игра должна работать стабильно в обычном программном режиме, просто потенциально с меньшей частотой кадров или упрощёнными эффектами. Следовательно, реализация должна предусматривать плавную деградацию качества графики или частоты кадров без потери основной функциональности.

2. Ограничения по инструментам:

- **Использование Pygame (Python):** Библиотека Pygame базируется на SDL (Simple DirectMedia Layer) и

предоставляет кроссплатформенный функционал для ввода, вывода звука, рендеринга изображений и обработки ивентов. Однако, она не предлагает полноценного 3D-ускорения или сложной графической логики. Поэтому визуальные эффекты должны быть реализованы через спрайты, статические и анимированные текстуры, простые шейдинговые приёмы в рамках возможностей Pygame. Для динамической оптимизации можно кэшировать спрайты, минимизировать количество поверхностей, оптимизировать частоту перерисовки.

- **Зависимость от SDL:** SDL обеспечивает низкоуровневый доступ к графической системе, устройствам ввода (клавиатура, геймпад, мышь) и системному таймеру. Любые ограничения SDL (например, особенности работы в оконном режиме или полноэкранном режиме на разных ОС, определённый набор поддерживаемых форматов изображений и аудио) напрямую влияют на функциональность игры.

3. Ограничения по разрешению экрана и системным требованиям:

- **Размеры экранов:** Игра должна корректно работать при различных разрешениях экрана, начиная с 1280x720 и выше. Возможны адаптивные масштабирования интерфейса и спрайтов для сохранения читаемости и визуальной целостности на мониторах с разным DPI. При очень низких разрешениях или нетипичных соотношениях сторон может потребоваться обрезка или адаптация игрового поля.
- **Минимальные системные требования:** Пример базовой конфигурации:
 - Процессор: любой современный двухъядерный CPU с тактовой частотой ~2 ГГц
 - ОЗУ: 4 ГБ и выше
 - Графика: встроенная или начального уровня дискретная карта, поддерживающая OpenGL 2.0 или выше (для возможной аппаратной оптимизации)
 - ОС: Windows 10, современный дистрибутив Linux или macOS 10.13+
- **Управление (клавиатура, геймпад):** Основные действия (прыжок, движение, атака, замедление времени) должны быть доступны через стандартную клавиатуру. Дополнительная поддержка геймпада может расширить аудиторию, но это добавляет тестовые сценарии и необходимость калибровки. Пользователи смогут переназначать клавиши, если это будет реализовано, для удобства и совместимости с нетипичными устройствами ввода.

Перечисленные выше ограничения задают рамки, в которых будет функционировать “Neon Runner”. Разработчики и дизайнеры смогут использовать информацию о производительности, аппаратных требованиях, возможных режимах рендеринга и поддерживаемых платформах, чтобы принимать обоснованные решения и гарантировать оптимальный баланс между функциональностью, качеством графики и стабильностью работы игры.

2.5 Допущения и зависимости

Разработка и функционирование игры “Neon Runner” базируется на ряде предположений, которые определяют качество конечного продукта и стабильность его исполнения. Понимание этих допущений и зависимостей позволит команде разработки своевременно выявлять риски, принимать превентивные меры, а также определить приоритеты в планировании.

1. Зависимость от графических ассетов:

- **Спрайты и анимации:** Предполагается наличие полного набора высококачественных 2D-спрайтов для персонажа, врагов, окружения, интерфейса и визуальных эффектов. Анимации должны быть оптимизированы по количеству кадров и размеру текстур, чтобы не допустить избыточного потребления видеопамати и снижения частоты кадров. Предполагается, что ассеты будут созданы заранее или в согласованные сроки, предоставлены в стандартных форматах (PNG, возможно с использованием спрайт-листов), обеспечивая лёгкую интеграцию в игру через Pygame.
- **Эффекты и слои освещения:** Зависимость от заранее подготовленных эффектов пост-обработки (световых вспышек, свечений) минимальна из-за ограниченных возможностей Pygame по сложному шейдингу. Однако эффекты могут быть имитированы с помощью заранее отрендеренных анимированных спрайтов или наложения полупрозрачных слоёв. Предполагается, что эти ресурсы будут доступными и корректно оптимизированными (сведены к минимальному необходимому размеру) до этапа финальной интеграции.

2. Звуки и музыка:

- **Звуковые эффекты:** Игра полагается на набор звуков (шаги, прыжки, стрельба, удары, звуки попадания, сбор аптечек, сигналы тревоги), которые должны быть заранее подготовлены в форматах сжатия без существенной потери качества (например, OGG или WAV) и оптимизированы для быстрой загрузки.
- **Музыкальное сопровождение:** Саундтрек (либо короткие музыкальные петли, либо полноформатные треки) создаёт атмосферу киберпанка. Предполагается, что музыка будет предоставлена в подходящих форматах (OGG, MP3), а её исполнение и адаптация к игровым ситуациям (например, смена трека при встрече с боссом) будут согласованы с дизайнерами.

3. Стабильность фреймрейта и отсутствие аппаратных проблем:

- **Допущение о стабильной работе на рекомендованной конфигурации:** Предполагается, что игра будет функционировать при стабильной частоте кадров (около 60 FPS) на ПК со спецификациями, определёнными в разделе ограничений (см. 2.4). На этих системах не должно возникать критических просадок фреймрейта или аномалий при отрисовке.
- **Отсутствие аппаратных проблем:** Делается предположение, что пользовательская система не имеет аппаратных неисправностей, конфликтов драйверов или иных проблем, которые могут вызвать сбои. Если такие ситуации возникнут, предполагается, что это будет выходить за рамки ответственности разработчиков игры.
- **Предсказуемость окружения:** Предполагается, что целевая среда (операционные системы, установленная версия Python, корректная конфигурация звуковой подсистемы, наличие необходимых шрифтов) будет стабильной, без неожиданных ограничений или нарушений совместимости.

4. Зависимость от интеграционных тестов и прототипов:

- Зависимость от ранних прототипов уровней, ИИ врагов и UI необходима для выверки баланса и производительности. Допускается, что на момент финальной сборки ассеты и игровой код пройдут через серию тестирований, позволяющих своевременно выявить узкие места (например, слишком большие текстуры или неэффективный цикл обновления логики).

5. Зависимость от библиотеки Pygame и SDL:

- Предполагается, что используемые версии Pygame и SDL стабильны, проверены и не содержат критических багов, препятствующих реализации функционала. Выход новых версий этих библиотек не должен принципиально ломать совместимость, но если это случится, обновление или фиксированная версия зависимостей будет рассматриваться как необходимая мера.

В совокупности данные допущения и зависимости создают контекст, в котором будет реализована и протестирована игра. Отклонение от перечисленных предпосылок может потребовать адаптации кода, ассетов или производственного процесса, чтобы обеспечить соответствие конечного результата целям и спецификациям.

3. Системные возможности (функциональные требования, functional requirements)

Для каждой функциональности определить приоритет, входные и выходные условия.

3.1 Игровой мир и окружение

Игровой мир “Neon Runner” представляет собой вертикально ориентированное пространство внутри и снаружи корпоративной башни Wallace. Основным фокусом является поступательное продвижение игрока вверх, по разнообразным архитектурным элементам и техническим конструкциям. Все аспекты окружения должны быть логично связаны как визуально, так и геймплейно, обеспечивая интуитивную навигацию и постоянный вызов для игрока.

Формирование уровней (вертикальных секций здания)

Уровни в “Neon Runner” состоят из последовательности вертикальных секций. Каждая секция может быть интерпретирована как отрезок пути от одной “этажной” платформы или контрольной точки до следующей. Отдельные секции выполняют различные функции:

- **Общая структура уровней.**
 - Уровни представляют собой ансамбль вертикальных коридоров, шахт, наружных фасадов, соединённых платформами, выступами, трубами и балконами.
 - Уровни могут чередоваться по стилю (внутренние инженерные отсеки, внешние участки здания, полуоткрытые сервисные зоны) для придания разнообразия и различной сложности паркура.
- **Модульность и процедурность (опционально).**
 - Возможна модульная структура, где каждый вертикальный сегмент выбирается из заранее подготовленных блоков.
 - Это позволяет повторно использовать конфигурации препятствий и варьировать их расположение, снижая предсказуемость и повышая интерес к повторным прохождениям.
- **Балансирование сложности.**
 - Нижние секции содержат более простые структуры, низкое расположение аптечек и минимальный набор врагов.
 - По мере подъёма сложность возрастает: платформы становятся реже, расстояния между ними больше, а враги и препятствия располагаются так, чтобы потребовать от игрока более точного времени реакции и стратегического использования ресурсов (замедление времени, оружие, аптечки).

Интерактивные элементы (платформы, трубопроводы, технические ниши)

Окружающая среда насыщена различными объектами, с которыми игрок может вступать в интеракцию. Они формируют “скелет” игрового пространства и влияют на динамику прохождения:

Элемент	Описание	Геймплейная роль	Пример взаимодействия
Платформы	Статичные или движущиеся поверхности для прыжков и стояния	Базовая поддержка для перемещения вверх	Игрок прыгает с одной платформы на другую
Трубопроводы	Вертикальные или горизонтальные трубы, за которые можно зацепиться или по которым можно скользить	Альтернативные пути, точки сохранения усилий	Игрок цепляется за трубу, чтобы передохнуть или выбрать путь к аптечке
Технические ниши	Углубления в стенах, подсобные отсеки	Спрятанные ресурсы (аптечки, батареи), временное укрытие	Игрок прячется от дрона или находит аптечку, скрытую от основного маршрута
Лифтовые шахты	Вертикальные каналы, в которых можно воспользоваться движущимися платформами или лифтами	Быстрое вертикальное продвижение, но с риском наткнуться на опасности	Игрок пробирается через шахту, избегая движущиеся механизмы
Разрушаемые объекты	Хрупкие панели, стеклянные перегородки, которые можно разбить ударом	Открытие альтернативных путей, доступ к аптечкам	Игрок ломает панель дубинкой и находит спрятанный ресурс

Использование этих интерактивных элементов даёт множество вариантов прохождения, позволяя игроку решать: идти ли самым очевидным, но опасным маршрутом или же рискнуть временем, чтобы обойти препятствие и найти дополнительные ресурсы.

Визуальная и функциональная связность окружения

- **Единый стиль и кодировка информации.**
 - Окружение оформлено в киберпанковом стиле: неоновые акценты, металлические конструкции, сетчатые помосты.
 - Опасные элементы (электрифицированные платформы, лазерные барьеры) подсвечены соответствующими цветами, чтобы игрок мог интуитивно распознавать угрозу.
- **Читаемость уровней.**
 - Ресурсы (аптечки, батареи) выделены контрастными оттенками, позволяющими игроку без труда отличить полезный объект от фона.
 - Враги и препятствия стратегически расположены таким образом, чтобы игрок мог заранее планировать прыжки и манёвры.

Итоговая роль игрового мира

Игровой мир и окружение являются не только декорацией, но и активным участником игрового процесса. Вертикальное пространство задаёт ритм геймплея, сложная структура платформ и интерактивных элементов формирует динамику перемещения и принятия решений, а визуально связанная среда помогает игроку быстро понимать ситуацию и адаптироваться к меняющимся условиям.

3.2 Управление персонажем

Основой игрового процесса “Neon Runner” является чётко проработанная физика и отзывчивое управление персонажем. Игрок должен ощущать плавность и предсказуемость движений, сохраняя контроль над героем в условиях высокоскоростного вертикального геймплея. Ниже описаны ключевые аспекты системы управления, физики и обработки столкновений.

Ввод с клавиатуры

Управление персонажем предполагается с помощью стандартных клавиш направления и функциональных кнопок. Возможна поддержка геймпада, но основным считается клавиатурный ввод.

Действие	Предлагаемые клавиши	Описание
Движение влево	← или A	Персонаж перемещается влево с базовой скоростью, зависящей от параметров физики.
Движение вправо	→ или D	Персонаж перемещается вправо. Аналогично влево, но в противоположном направлении.
Прыжок	Пробел или W	Персонаж совершает прыжок. Сила прыжка задана физическими параметрами и не меняется динамически.
Атака	J или K (пример)	Активация ближнего или дальнего оружия. Можно ввести разную раскладку для разных типов атак.
Замедление времени	Shift или Ctrl	При удержании включается режим замедления времени, при отпускании — прекращается.

Все клавиши могут быть при необходимости переназначены в настройках (опционально).

Физика движения

Персонаж подчиняется простым, но аккуратно сбалансированным физическим законам. Параметры задаются числовыми константами, которые можно подбирать при тестировании для оптимального геймплея.

- **Сила прыжка (Jump Force).** Определяет, насколько высоко игрок может прыгнуть. Обычно выражается в пикселях или условных единицах. Например: $\text{Jump_Force} = 300$, что означает подъём на заданное число пикселей при нажатии прыжка.
- **Гравитация (Gravity).** Постоянная сила, тянущая персонажа вниз. Может иметь значение порядка $\text{Gravity} = 1000$ (условные единицы/с²), что приводит к равномерному ускорению падения.
- **Горизонтальная скорость (Horizontal Speed).** Базовая скорость передвижения по оси X. Например, $\text{SpeedX} = 200$, означающая 200 пикселей в секунду при удержании клавиши направления.
- **Максимальная скорость вертикального движения (Vertical Limits).** Можно ограничить максимальную скорость падения для большей управляемости. Например, $\text{MaxFallSpeed} = 800$.

Аналогично можно ввести параметры для ускорений, замедлений, скольжений по трубам, если они потребуются.

Обработка коллизий

Коллизии – важнейший аспект геймплея, отвечающий за реалистичное взаимодействие персонажа с окружением и врагами. Правильная обработка столкновений обеспечивает предсказуемость и отсутствие “застреваний” в геометрии уровней.

- **Столкновения с платформами и стенами:**
 - Вертикальные и горизонтальные проверки: при каждом обновлении кадра игра вычисляет новую потенциальную позицию персонажа на основе скорости и направления движения.
 - Если путь пересекает невходимую поверхность (платформа, стена), позиция корректируется до границы препятствия, скорость в соответствующем направлении обнуляется или изменяется.
 - При приземлении на платформу гравитация перестаёт увеличивать скорость вниз. Персонаж переходит в состояние “на земле”, что позволяет вновь совершать прыжок.
- **Столкновения с врагами:**
 - Враги представляют собой движущиеся или статичные объекты с хитбоксами. При пересечении хитбокса персонажа и врага происходит событие удара.
 - Если персонаж атакует (ближний бой или выстрел) и в этот момент хитбоксы оружия и врага пересекаются, враг получает урон. Аналогично, если враг атакует и достигает хитбокса персонажа, игрок теряет здоровье или время жизни.
 - Логика обработки задаётся приоритетами: если у персонажа есть активный удар, он может прервать атаку врага. Если враг защищён или атакует первым, игрок получает урон.

Пример системы проверки коллизий

Алгоритм на каждом кадре (упрощённо):

1. Рассчитать потенциальную новую позицию персонажа:

```
newX = currentX + speedX * deltaTime
```

```
newY = currentY + (verticalSpeed + gravity * deltaTime) * deltaTime
```

2. Проверить коллизии по оси X:

- Если newX пересекает стену, скорректировать newX до границы препятствия, сбросить speedX до 0.

3. Проверить коллизии по оси Y:

- Если newY касается платформы снизу, установить newY на верхнюю поверхность платформы, сбросить verticalSpeed до 0, пометить состояние "onGround = true".
- Если newY пересекает опасные объекты (шпы, электричество), применить урон игроку.

- 4. Обновить позицию персонажа до (newX, newY).
- 5. Проверить пересечение хитбоксов с врагами:
 - Если игрок атакует и совпадает с врагом по координатам хитбоксов оружия, нанести урон
↪ врагу.
 - Если враг атакует или просто соприкасается с персонажем, применить соответствующий урон к
↪ игроку.
- 6. Обновить состояние анимаций и UI (например, стрелка, показывающая направление движения,
↪ подсветка для замедления времени).

Данные схематично описанные механизмы обеспечивают предсказуемое и отзывчивое управление, где любой нажатый игроком ввод напрямую отражается в движении персонажа, а физические параметры и алгоритмы коллизий гарантируют сбалансированный игровой опыт.

3.3 Взаимодействие с “аптечками”

“Аптечки” являются критическим ресурсом для продления времени жизни репликанта. Игроку необходимо их собирать, чтобы удерживать таймер на приемлемом уровне, балансируя между быстрым продвижением вверх и поиском дополнительных ресурсов.

Условия появления “аптечек” на уровнях

“Аптечки” размещаются с учётом сложности и частоты их попадания на глаза игроку:

Тип расположения	Описание	Цель
Открытые площадки	“Аптечки” находятся на видимых платформах	Обеспечить базовый уровень поддержки, стимулировать постоянное движение вверх
Труднодоступные ниши	“Аптечки” спрятаны за разрушаемыми панелями или в узких вертикальных шахтах	Поощрять исследование и использование умений (замедление времени, оружие) для доступа
Лабораторные секции	Специальные мини-лаборатории между этажами	Позволить получить более ценные “аптечки” за дополнительный риск или решение головоломок

Частота появления “аптечек” настроена так, чтобы игрок был вынужден активно их искать и принимать решения о рисках.

Механика продления таймера жизни

Собранная “аптечка” немедленно добавляет дополнительное время к текущему таймеру персонажа.

Два вида “аптечек”:

- Обычные “аптечки”: дают +30 секунд к таймеру. Встречаются чаще.
- Редкие “аптечки”: дают +60 секунд к таймеру. Появляются реже, обычно в труднодоступных местах или лабораторных секциях.

Пример: если у игрока остаётся 10 секунд до истечения жизни, сбор обычной “аптечки” продлит время до 40 секунд, а редкой — до 70 секунд.

Ограничение на максимальную длительность жизни

Чтобы не допустить накопления слишком большого резерва времени, вводится максимальное ограничение продолжительности:

Параметр	Значение (пример)	Комментарий
Максимальное время жизни	120 секунд	Игрок не может превысить этот лимит времени

Если суммарный эффект “аптечек” превысит лимит, время остаётся на максимуме, поддерживая постоянное напряжение и не давая игроку накапливать бесконечный запас.

Итоговая роль аптечек

Аптечки поддерживают баланс между риском и вознаграждением. Частые 30-секундные аптечки помогают поддерживать стабильный прогресс, тогда как редкие 60-секундные аптечки позволяют игроку почувствовать существенный выигрыш времени, но требуют б’ольших усилий и умения. В итоге система аптечек формирует динамичный и напряжённый опыт, где управление временем жизни становится критически важной частью стратегии прохождения.

3.4 Взаимодействие с “врагами”

Враги в “Neon Runner” являются ключевым фактором, формирующим динамику геймплея и уровень вызова. Они вынуждают игрока применять стратегию, использовать оружие, замедление времени и предвидеть поведение противников. Ниже описаны основные типы врагов, их характеристики, а также условия поражения игрока.

Летающие дроны

“Летающие дроны” — это автономные охранные единицы, способные патрулировать вертикальные коридоры и реагировать на присутствие игрока.

Характеристика	Описание
Зона обнаружения	Дрон активируется при входе игрока в определённый радиус или прямоугольную область
Тип атак	Лазерные выстрелы по прямой или краткие залпы по дуге
Урон	При попадании лазерного импульса игрок теряет часть здоровья или времени жизни
Поведение при обнаружении	Дрон переходит в активную фазу: следует за игроком, пытаясь удержать удобную позицию для стрельбы

Игроку необходимо оценить траекторию движения дрона и временно или пространственно обойти его, применить оружие или замедление времени для уверенного уничтожения.

Патрульные на платформах

“Патрульные” — гуманоидные или кибернетические охранники, перемещающиеся по фиксированным маршрутам на горизонтальных поверхностях.

Характеристика	Описание
Маршруты	Предопределённые пути движения (влево-вправо), останавливаются на краях платформ
Поведение при обнаружении	При визуальном контакте или вхождении игрока в зону зрения патрульный прекращает патруль и начинает стрельбу или бежит навстречу

Характеристика	Описание
Боевые паттерны	Могут использовать огнестрельное или шоковое оружие, периодически укрываться за объектами
Стратегия для игрока	Игрок может использовать замедление времени, прыгать над патрульным, атаковать дистанционно или выманить его на невыгодную позицию

Мини-боссы “бегущие по лезвию”

Мини-боссы — это особые враги, обладающие увеличенным здоровьем, более сложными паттернами поведения и особыми атаками.

Характеристика	Описание
Уникальные атаки	Высокоточная винтовка, самонаводящиеся дроны, усиленные рукопашные удары
Повышение сложности	Мини-босс может менять стратегию по мере снижения здоровья, усиливать своих дронов или переключаться на более агрессивный паттерн
Награда за победу	После уничтожения мини-босса игрок может получить редкую “аптечку”, батарею для оружия или доступ к новой вертикальной секции здания
Тактическая глубина	Игроку придется сочетать все доступные механики: уход от огня, точные прыжки, замедление времени, оружие ближнего и дальнего боя

Условия поражения

Игрок проигрывает в двух случаях:

- 1. **Исчерпание времени жизни:**
Если таймер жизни опускается до нуля и игрок не успевает собрать новую “аптечку”, персонаж прекращает существование, и игра заканчивается.
- 2. **Критический урон:**
Если количество попаданий от врагов достигает критического порога (например, при отсутствии брони или достаточного запаса здоровья/времени), персонаж умирает. Это стимулирует игрока избегать прямых столкновений или использовать механики обхода и замедления времени.

Итоговая роль врагов

Враги создают основу для тактической части геймплея. Каждый тип противника, от простых дронов до мини-боссов, требует индивидуального подхода, управления ресурсами и быстрой реакции. Игрок должен анализировать ситуацию, использовать окружение, применять замедление времени и оружие, чтобы одержать верх и продвинуться вперед.

3.5 Оружие и предметы

В “Neon Runner” оружие и специальные приспособления помогают игроку противостоять врагам, преодолевать сложные участки уровней и эффективно управлять пространством. Каждое средство имеет свои ограничения и условия использования, стимулируя тактическое планирование и продуманные действия.

Электро-дубинка

“Электро-дубинка” — оружие ближнего боя, применяемое для быстрой нейтрализации врагов на короткой дистанции.

Характеристика	Описание
Дальность атаки	Очень короткая (порядка 1-2 единиц расстояния от игрока)
Урон	Средний. Достаточно удара или двух для нейтрализации стандартного патрульного
Ограниченные заряды	Допускается концепция “зарядов” или “энергии” дубинки. Каждый удар тратит часть заряда. Заряд можно пополнять, находя редкие батареи или временно подождав
Анимация и эффект	При ударе вспышка неоновой света и электрический разряд, кратковременно оглушающий противника

Использование “электро-дубинки” целесообразно, когда нужно бесшумно и быстро устранить цель, сэкономив патроны для дальнобойного оружия.

Импульсный пистолет

“Импульсный пистолет” — дальнобойное оружие, обеспечивающее безопасность, не подходя близко к врагам.

Характеристика	Описание
Дальнобойность	Средняя. Лучи пистолета распространяются на 2-3 экрана по вертикали или горизонтали, но имеют небольшую задержку перед попаданием
Перезарядка	После нескольких выстрелов пистолету требуется короткий интервал перезарядки (например, после 5 залпов — 1 секунда без стрельбы)
Ресурс патронов	Пистолет использует “патроны” или “энергию”. Пополняются, находя батареи в уровнях. Количество патронов ограничено
Анимация	Яркий неоновый луч, короткая вспышка в точке попадания. При попадании по врагу — визуальный эффект искр или электронного сбоя

“Импульсный пистолет” полезен против дронов или патрульных, находящихся на расстоянии. Он даёт игроку стратегический выбор: осторожная дистанционная тактика или ближний контакт для экономии патронов.

Крюк-кошка

“Крюк-кошка” — устройство, позволяющее быстро подниматься на трудно достижимые платформы и обходить опасные зоны.

Характеристика	Описание
Условия использования	Механизм требует временной перезарядки после каждого использования (например, 3-5 секунд), или ограниченный заряд, восполняемый со временем
Анимация	При нажатии соответствующей клавиши крюк выстреливает вверх, зацепляется за выступ, и игрок молниеносно поднимается на несколько десятков пикселей
Ограничение по высоте	Дальность крюка ограничена (например, не более 200 пикселей вверх). Если цель выше, использовать крюк нельзя
Стратегическая роль	Позволяет экономить время на трудных прыжках, быстро уходить от врагов или добираться до спрятанных “аптечек”. При этом неправильное использование может привести к затрате ресурса попусту и оставит игрока без поддержки в критический момент

Применение “крюка-кошки” делает вертикальный геймплей более вариативным, позволяя игроку выбирать альтернативные маршруты и подбирать стратегию продвижения, учитывая расположение врагов и ресурсов.

Все перечисленные средства дополняют игровой баланс и дают игроку инструмент для решения сложных ситуаций. Они ограничены ресурсами, перезарядкой или физическими параметрами, что не позволяет использовать их бездумно и заставляет проявлять тактическое мышление и мастерство.

3.6 Механика “замедления времени”

Механика “замедления времени” позволяет игроку на короткое время снизить общую динамику игрового окружения, давая больше возможностей для точного маневрирования, прицеливания и избегания опасностей. Это важный стратегический инструмент, который следует применять осознанно.

Потребление ограниченного ресурса

Включение “замедления времени” расходует специальный ресурс (например, “энергия” или “внутренний заряд”), который восстанавливается со временем или при сборе определённых предметов.

Характеристика	Описание
Тип ресурса	“Энергия замедления” или “Временной заряд”
Объём ресурса	Ограниченный запас (например, 100 единиц), который уменьшается с каждой секундой замедления
Пополнение	Редкие предметы на уровнях или постепенная регенерация за время, проведённое без замедления

Когда ресурс исчерпывается, замедление автоматически отключается, вынуждая игрока грамотно управлять этим преимуществом.

Эффект на скорость движения

При включённом замедлении времени скорость всех движущихся объектов снижается, но сам игрок сохраняет относительную манёвренность. Это позволяет ему:

- **Движение персонажа:** Немного замедляется, но всё ещё быстрее относительно замедленных врагов и снарядов, обеспечивая лучшее управление прыжками и позиционированием.
- **Скорость врагов:** Враги и их атаки также замедляются, давая больше времени на реакцию.
- **Снаряды и элементы окружения:** Лазерные выстрелы дронов, падающие объекты, движущиеся платформы — всё идёт медленнее, упрощая точный тайминг действий.

Критические моменты применения

“Замедление времени” должно использоваться в ситуациях, где без точного расчёта игрок рискует получить урон или упустить шанс на продвижение:

- **Точные прыжки:** При сложных вертикальных секциях, требующих идеальной точности, замедление даёт возможность выждать идеальный момент для прыжка.
- **Избегание атак врагов:** При массированных атаках дронов или выстрелах мини-боссов игрок может активировать замедление, чтобы аккуратно проскользнуть между снарядами.
- **Сбор труднодоступных ресурсов:** Если аптечка или редкий предмет расположен в зоне с ограниченным временем доступа (например, разрушающаяся платформа), замедление времени помогает безопасно достать ресурс, не рискуя упустить момент.

Таким образом, механика “замедления времени” создаёт дополнительные стратегические возможности для игрока. Она не даёт постоянного превосходства, так как ограничена ресурсом, но при правильном использовании может стать ключом к успешному преодолению критических ситуаций и повышению эффективности игрового процесса.

3.7 Прогресс и цели

Система прогресса в “Neon Runner” выстроена вокруг концепции постепенного восхождения к вершинам здания корпорации Wallace и преодоления череды испытаний, предлагаемых вертикальными уровнями, ограниченным временем и противостоящими врагами. Игрок имеет несколько ключевых целей, каждая из которых тесно связана с геймплеем и сюжетом.

Основные цели игрока

Цель	Описание	Значение для геймплея
Достигнуть верхних уровней здания	Подниматься по этажам, преодолевая препятствия и врагов	Ядро игрового процесса: вертикальный платформенный прогресс
Собрать достаточно “аптечек”	Поддерживать или увеличивать таймер жизни персонажа	Ведёт к стратегическому использованию времени и ресурсов
Победить финального босса	Сразиться с прототипным охранителем в верхних уровнях	Кульминационная схватка, проверяющая навыки и снаряжение игрока
Активировать генетическую модификацию	Использовать специальную капсулу в лаборатории Wallace	Финальный сюжетный штрих: снятие генетического ограничения

Динамика прогресса

- Вертикальный рост сложности:**
По мере продвижения вверх уровни становятся более комплексными, враги и препятствия — опаснее. Собирая “аптечки”, игрок приобретает тактическую свободу для борьбы с возрастающей сложностью.
- Промежуточные точки и контрольные зоны:**
Отдельные этажи или технические платформы могут выступать как точки относительного отдыха, возможность подсчитать ресурсы, оценить оставшееся время и подготовиться к следующему прорыву.
- Сюжетная мотивация:**
По мере продвижения к верхним уровням игрок всё ближе к освобождению от ограничений, заложенных корпорацией Wallace. Это создаёт эмоциональную вовлечённость, мотивацию преодолевать рискованные ситуации и искать оптимальную стратегию.

Завершение игры

- Победа над финальным боссом:**
Эта встреча — кульминация игрового процесса. Босс использует расширенный набор атак и тактик, требуя от игрока мастерского владения оружием, замедлением времени и умения экономить “аптечки”.
- Активирование генетической модификации:**
После победы игрок получает доступ к генетической лаборатории. Активация модификации — финальный сюжетный акт, снимающий временные ограничения. Это символическое вознаграждение, открывающее новую жизнь персонажу.

Результат

Когда все цели достигнуты, игрок получает полноценный опыт: он не только преодолел вертикальные испытания и врагов, но и освободился от искусственных генетических оков. Этот финал связывает геймплейные аспекты (прогресс, ресурсы, тактику боя) с нарративными элементами, придавая завершённость и смысл всем принятым в процессе прохождения решениям.

3.8 Финальный босс

Кульминацией прохождения “Neon Runner” становится противостояние с финальным боссом — прототипным ультра-репликантом, стоящим на страже лаборатории Wallace. Эта схватка представляет собой сложный многофазный бой, требующий от игрока мастерства в управлении ресурсами, точного тайминга и глубокого понимания игровых механик.

Состояния босса и типы атак

Финальный босс обладает несколькими фазами (состояниями), каждая из которых активируется при снижении его уровня здоровья до определённой отметки:

Фаза/Состояние	Описание	Типы атак и поведение
Начальная фаза	Босс анализирует действия игрока, совершает точечные атаки	Стрельба из винтовки по прямой, вызов нескольких дронов для поддержки
Средняя фаза	После потери части здоровья босс ускоряется, меняет паттерн атак	Могут появиться самонаводящиеся импульсы, комбинированные атаки с дронами
Критическая фаза	На последнем отрезке здоровья босс становится крайне агрессивным	Частые прыжки между платформами, зонирование пространства, перекрёстный огонь

Каждая фаза требует от игрока адаптации тактики: использование замедления времени для уклонения от массированных атак, экономия патронов и аптечек, выбор между ближним боем или стрельбой на расстоянии.

Реакция на действия игрока

Босс реагирует на действия игрока динамически:

- Попадания по боссу:**
Каждое успешное попадание уменьшает здоровье босса. При критических попаданиях в уязвимые моменты (например, в окне между атаками) босс может кратковременно отступить или изменить позицию.
- Защитное поведение:**
Если игрок слишком эффективно атакует на дальнем расстоянии, босс может создавать энергетические щиты или вызывать дронов, усложняя ситуацию и вынуждая игрока менять тактику.
- Взаимодействие с окружением:**
Босс может разрушать определённые элементы окружения, лишая игрока безопасных зон или создавая новые барьеры.

Условия победы/поражения

- Победа игрока:**
Игрок побеждает, доведя здоровье босса до нуля. При этом запускается сценка победы, в которой босс прекращает сопротивление и открывается доступ к финальной лаборатории.
- Поражение игрока:**
Если таймер жизни достигает нуля, или игрок получает критический урон от атак босса и его союзников, игра заканчивается. Игрок возвращается к предыдущим контрольным точкам или начинает финальную битву заново.

Триггер финальной сцены после победы

После победы над боссом активируется специальный триггер, запускающий финальную сцену, в которой игрок получает доступ к генетической модификации. Эта сцена завершает сюжетную линию и подтверждает достижение главной

цели — освобождение репликанта от генетических ограничений. Таким образом, победа над боссом становится не просто испытанием навыков, но и сюжетным катарсисом, завершающим игровой опыт.

3.9 Финальная сцена/Концовка игры

Финальная сцена — логическая и эмоциональная кульминация всего пройденного пути. После победы над финальным боссом и доступа к генетической модификации игрок становится свидетелем завершения истории, получая заслуженное чувство удовлетворения и свободы.

Анимация трансформации репликанта

После активации генетической модификации репликант входит в специальную капсулу. Проигрывается анимация, в которой:

- **Стадии трансформации:**
 1. Персонаж в капсуле подсвечивается неоновым светом.
 2. Голографические интерфейсы анализируют ДНК и отражают процесс “перезаписи” генетического кода.
 3. На теле репликанта проступают новые световые узоры, символизирующие свободу от ограничений.
- **Визуальные эффекты:** Яркие неоновые вспышки, мягкая анимация постепенной смены цвета и фактуры на модели персонажа, намекающая на стабильность и гармонию. Камера делает плавный наезд, фиксируясь на счастливым, обновлённом лице репликанта.

Отображение титров

После завершения трансформации сцена смягчается в более спокойный тон. На фоне панорамы ночного мегаполиса, где свет неона смешивается с утренней дымкой, начинают плавно появляться титры:

- **Содержание титров:**
 - Имена разработчиков (художников, программистов, дизайнеров).
 - Благодарности спонсорам, бета-тестерам и вдохновителям проекта.
 - Упоминание использованных библиотек (Pygame, SDL) и других инструментов.
- **Стиль показа титров:** Титры скроллят плавно сверху вниз или появляются в “мигающем неоновом” стиле. На фоне звучит атмосферный звуковой трек, завершающий киберпанк-историю.

Сценарий завершения игры и возможности перезапуска

После прохождения титров игроку предоставляются варианты действий:

- **Возврат в главное меню:** Игрок может перейти к стартовому экрану, выбрать другой режим сложности, начать новую игру или просмотреть дополнительные материалы (например, галерею концепт-арта или таблицы рекордов).
- **Новая игра+:** (опционально) Возможность начать прохождение заново с сохранением некоторых достижений (например, с альтернативными костюмами, дополнительными ресурсами или повышенной сложностью), поощряя повторные попытки и исследование пропущенных путей.
- **Завершение:** Если пользователь решит покинуть игру, она завершается корректно, сохраняя прогресс и достижения. При следующем запуске игрок сможет загрузить последний чекпоинт или начать с нуля.

Таким образом, финальная сцена не только подводит итог путешествию персонажа, но и логично завершает игровой цикл, предлагая игроку дальнейший выбор — насладиться послевкусием от победы или вернуться в мир “Neon Runner” для новых испытаний.

4. Нефункциональные требования

4.1 Производительность

Производительность игры “Neon Runner” — ключевой фактор, влияющий на общее качество пользовательского опыта. Все аспекты — от отклика на нажатия клавиш до гладкой отрисовки спрайтов — должны быть оптимизированы для обеспечения комфортного геймплея.

Целевой фреймрейт

Игровой процесс ориентирован на стабильную частоту кадров:

- **Целевой фреймрейт:** 60 FPS (кадров в секунду).
- **Допустимые отклонения:** Кратковременные просадки до 50-55 FPS считаются приемлемыми, при условии, что они не сопровождаются визуальными артефактами или задержками в управлении.

Чтобы поддерживать стабильный FPS, необходимо оптимизировать код обновления логики, обработку столкновений и рендеринг. Это достигается путём правильной организации игрового цикла и использования эффективных структур данных.

Задержка ввода

Реакция игры на действия игрока должна быть практически мгновенной:

- **Максимальная задержка ввода:** не более 50 мс с момента нажатия клавиши до реакции персонажа на экране.
- **Цели оптимизации:**
 - Сократить количество дорогостоящих операций в основном цикле (рендер, физика).
 - Гарантировать, что считывание ввода (клавиатура, геймпад) происходит на ранних этапах каждого кадра.

Такой уровень отзывчивости создаёт иллюзию прямого контроля и повышает доверие к игровому процессу.

Оптимизированный рендер спрайтов

Визуальное оформление “Neon Runner” базируется на использовании множества 2D-спрайтов. Оптимизации необходимы, чтобы избежать перегрузок видеопамати и снизить нагрузку на CPU/GPU:

Приём оптимизации	Описание
Предварительная загрузка	Все основные спрайты (персонаж, враги, окружение) загружаются при старте уровня, минимизируя подгрузки в реальном времени
Спрайт-листы (Sprite Sheets)	Объединение кадров анимаций в один файл для сокращения количества операций загрузки и отображения
Пулинг объектов	Повторное использование уже созданных объектов (спрайтов, частиц) для избежания частого выделения/освобождения памяти
Ограничение разрешения текстур	Использование текстур оптимального разрешения, без избыточных пикселей, чтобы снизить нагрузку на видеоподсистему

Общие требования

Соблюдение требований к производительности, включающее стабильно высокий фреймрейт, минимальную задержку ввода и оптимизированный рендеринг, гарантирует плавный и отзывчивый игровой опыт. Эти меры повышают комфорт игрока и укрепляют общее впечатление от игры “Neon Runner”.

4.2 Надёжность

Надёжность игры “Neon Runner” — определяющий фактор стабильного и предсказуемого игрового опыта. Все компоненты, от подсистем физики до управления ресурсами, должны быть спроектированы и протестированы так, чтобы свести к минимуму вероятность критических ошибок, приводящих к сбоям или непредсказуемому поведению.

Минимизация багов в физике и коллизиях

Физическая модель игры обеспечивает реалистичное и предсказуемое перемещение персонажа и взаимодействие с окружением. Недопустимо, чтобы ошибки в расчётах или неверная обработка столкновений приводили к “застреванию” персонажа, проникновению сквозь стены или иным аномалиям.

- **Методы обеспечения надёжности:**

- Предварительное тестирование на макетных уровнях, где отрабатываются базовые сценарии прыжков, скольжений, падений.
- Использование чётко определённых алгоритмов коллизий и обновления координат: все физические перемещения должны происходить по шагам, а после каждого шага — проверка и коррекция позиции.
- Логирование аномалий (например, если при столкновении персонаж оказывается вне границ уровня, запись в лог-файл для дальнейшего анализа).

Работа без критических сбоев на целевой платформе

Игра должна стабильно работать в типовых условиях на поддерживаемых платформах (Windows, Linux, macOS), без неожиданных вылетов, зависаний или потери данных.

- **Принципы обеспечения стабильности:**

- Обработка ошибок ввода-вывода: корректная реакция на отсутствие шрифтов, ресурсов, попытки запуска в нестандартных разрешениях экрана.
- Использование устойчивых к сбоям механизмов загрузки ресурсов: проверка доступности файлов, fallback-пути при отсутствии некоторых ассетов.
- Регулярные стресс-тесты и нагрузочные тесты: проверка работы при повышенной интенсивности событий (много врагов, множественные эффекты).

Итого

Тщательной проработки игровой логики, физики и системы ввода-вывода, а также комплексного тестирования. Безупречная предсказуемость взаимодействия с окружением, стабильная работа без вылетов и минимальное число багов гарантируют, что игроки смогут сосредоточиться на игровом процессе, а не на технических проблемах.

4.3 Удобство использования (юзабилити)

Комфорт и понятность интерфейса являются приоритетом для привлечения и удержания игроков с разным уровнем подготовки. Важна возможность быстро освоить управление и получать чёткую обратную связь на действия.

- **Интуитивное управление:** Клавиши и кнопки, отвечающие за движение, прыжки, замедление времени и атаки, должны быть назначены логично (например, стрелки или WASD для перемещения, пробел для прыжка). Можно предусмотреть опцию переназначения клавиш и поддержку геймпадов для расширения удобства.
- **Обучающий уровень или подсказки:** В первых секциях игры могут быть ненавязчивые подсказки, помогающие освоить базовые действия: прыжки, сбор “аптечек”, применение оружия, использование замедления времени. Обучающие элементы должны быть интегрированы в геймплей, а не быть навязчивыми диалогами или статичными экранами.

- **Чёткая визуальная обратная связь:** Визуальные эффекты (вспышки при сборе “аптечки”, искры при попадании по врагу, молнии при ударе дубинкой) дают игроку мгновенное понимание результата своих действий. Индикаторы состояния (таймер жизни, ресурс замедления, количество патронов) должны быть чёткими, хорошо читаемыми и расположенными так, чтобы их можно было воспринимать периферийным зрением.

4.4 **Портируемость**

Поддержка нескольких операционных систем и гибкая настройка параметров экрана расширяют аудиторию игры и повышают удобство пользователей.

- **Возможность запуска на разных ОС (Windows, Linux, macOS):** Код должен быть кроссплатформенным, использовать библиотеки и инструменты, одинаково хорошо работающие в различных окружениях. Тестирование сборок на всех целевых платформах — обязательная часть процесса.
- **Гибкая настройка разрешения экрана:** Игрок должен иметь возможность запускать игру в разных разрешениях и режимах (окно, полный экран). Интерфейс и спрайты при этом масштабируются или перестраиваются так, чтобы сохранять читаемость и не искажать визуальный стиль.

4.5 **Масштабируемость**

Игра должна предусматривать возможность развития и расширения контента без серьёзных изменений в основном коде.

- **Возможность добавления новых уровней, врагов, предметов без переработки ядра:** Архитектура проекта и структура кода должны быть модульными. Новый контент (спрайты, конфигурации врагов, параметры уровней, варианты оружия) должен добавляться путём изменения или дополнения внешних файлов данных (JSON, CSV, YAML) или простым подключением новых модулей без кардинального переписывания логики.

Таким образом, расширение контента — это вопрос изменения настроек и ассетов, а не крупных переделок движка.

4.6 **Локализация**

Поддержка нескольких языков повышает доступность игры для международной аудитории.

- **Поддержка нескольких языков интерфейса (опционально):** Все текстовые элементы (меню, подсказки, описания предметов, диалоги) должны быть вынесены во внешние ресурсы, чтобы их можно было локализовать. Переключение языков выполняется автоматически или через меню настроек. Хотя эта функция опциональна, архитектура проекта должна предусматривать простой способ добавления новых языков, без изменений в исходном коде игровой логики.

Локализация позволит адаптировать игру под разные рынки, минимизируя затраты на интеграцию нового языкового пакета.

5. **Внешние интерфейсы**

5.1 **Пользовательский интерфейс (UI)**

Интерфейс должен быть минималистичным, но информативным, интегрируясь в киберпанковую эстетику игры. Главное правило — вся важная информация должна быть доступна игроку без отвлечения от основного процесса.

Основные элементы UI:

Элемент	Описание	Цель/Функция
Меню начала игры	Включает кнопки “Начать игру”, “Настройки”, “Выход”	Предоставляет игроку базовый навигационный интерфейс перед началом игры

Элемент	Описание	Цель/Функция
Меню настроек	Параметры звука, разрешение экрана, изменение клавиш	Позволяет игроку адаптировать опыт под свои предпочтения
Таблицы результатов	Отображение рекордов, времени прохождения, набранных очков	Стимулирует соревновательный интерес, даёт мотивацию к повторным попыткам
Индикатор времени жизни	Цифровой таймер или шкала, уменьшающаяся со временем	Даёт игроку понимание текущего запаса времени, стимулирует поиск “аптечек”
Индикатор количества “аптечек”	Числовой или графический индикатор	Показывает, сколько дополнительных ресурсов для продления жизни собрано
Индикатор ресурса замедления времени	Шкала или иконка, показывающая остаток “энергии”	Помогает планировать тактическое использование механики замедления времени

Обратная связь:

- Яркие вспышки при сборе “аптечки” или при ударах дубинкой.
- Лёгкая вибрация или цветное мерцание экрана при попадании по игроку.
- Динамические подсказки: при критическом снижении времени жизни интерфейс может подсвечивать ближайшие ресурсы или указывать на горячие клавиши.

5.2 Интерфейс звука

Звуковое оформление — важная часть атмосферы, помогающая ориентироваться в пространстве и событиях игры.

Основные звуковые эффекты:

Звуковой эффект	Описание	Функция
Прыжок	Короткий, лёгкий звук (“пружинистый” оттенок)	Подтверждает успешность действия, создаёт тактильное ощущение
Атака (дубинка, пистолет)	Электрический разряд, импульсный выстрел	Помогает ориентироваться в бою, различать тип атаки
Сбор “аптечки”	Мягкий неоновый импульс	Передаёт положительный отклик, подтверждает получение ресурса
Звуки попаданий	Искажения, искры, электронные сбои	Информируют о нанесении урона врагу или игроку

Фоновая музыка:

- Стиль: Киберпанк с неоновыми синтезаторными оттенками.
- Динамика: На ранних этажах — спокойные, атмосферные треки; по мере продвижения выше — более напряжённые.
- При столкновении с боссом: трек меняется на более ритмичный, агрессивный, подчёркивая опасность и важность момента.

5.3 Интерфейсы сохранения данных

Хотя “Neon Runner” может быть ориентирован на непрерывный геймплей, предусмотрена возможность сохранения прогресса или результатов.

Типы данных для сохранения:

Данные	Пример	Цель
Контрольные точки (чекпоинты)	Этаж или уровень, достигнутый игроком	Позволяют возобновить игру с определённого места, не начиная с нуля
Рекорды и очки	Лучшее время прохождения, максимальный счёт	Стимулирует возвращаться к игре, улучшать свои результаты
Настройки игрока	Громкость звука, раскладка клавиш	Повышает удобство пользования игрой и её индивидуальную адаптацию

Реализация (опционально):

- Файлы сохранений в простых форматах (JSON, YAML), расположенные в отдельной папке.
- Автоматическое сохранение при достижении ключевых этажей или после победы над мини-боссами.
- Возможность загрузить сохранение из главного меню.

Таким образом, внешние интерфейсы — UI, звук и сохранения — работают в комплексе, создавая удобную, понятную и атмосферную среду, в которой игрок может полноценно погрузиться в геймплей “Neon Runner”.

6. Атрибуты качества

6.1 Безопасность

Безопасность подразумевает отсутствие вредоносных элементов и корректную обработку внешних данных. Игра должна работать предсказуемо при любых стандартных вводах и не допускать компрометации системы.

Требования по безопасности:

Атрибут	Описание	Цель
Отсутствие вредоносного кода	Проверка и валидация всего внешнего кода и библиотек	Исключение внедрения вредоносных компонентов
Защита от некорректных файлов	Если игра загружает пользовательские данные (настройки, сохранения), нужен механизм проверки формата и корректности	Предупреждение ошибок при чтении повреждённых или неправильных файлов

При обнаружении некорректных данных игра должна работать по принципу “fail-safe”: пропустить некорректные файлы, заменить их на значения по умолчанию или уведомить пользователя.

6.2 Поддерживаемость

Поддерживаемость определяет лёгкость внесения изменений, улучшений и исправления ошибок без нарушения существующего функционала.

Требования по поддерживаемости:

Атрибут	Описание	Цель
Чистая архитектура кода	Использование модульного подхода, разделение логики по слоям (рендер, физика, AI), понятное именование переменных и функций	Упрощение сопровождения и расширения проекта
Документация к основным модулям	Комментарии в коде, wiki или отдельные файлы описания классов и функций	Быстрое вхождение новых разработчиков, снижение времени на поиск решений

Таким образом, любые улучшения или добавления контента (новые уровни, объекты, враги) могут быть реализованы без масштабной переработки базовой логики.

6.3 Тестируемость

Тестируемость означает, что игровой код и механики можно проверять автоматизировано или вручную без значительных усилий, позволяя своевременно выявлять и устранять ошибки.

Требования по тестируемости:

Атрибут	Описание	Цель
Возможность легко запускать тестовые уровни	Наличие упрощённого режима или параметра запуска, подгружающего специальные тестовые сцены	Быстрый отладочный цикл, проверка конкретных механик
Наличие логирования ключевых событий	Запись в файл или консоль информации о действиях, ошибках, прохождении уровней	Упрощает анализ и диагностику проблем, сокращая время на поиск источника ошибок

Благодаря этим мерам команда разработки сможет регулярно проверять качество сборок, контролировать стабильность и вовремя реагировать на возникающие проблемы.

7. Системные требования к окружению

7.1 Аппаратные требования

Аппаратные требования определяют минимальную и рекомендуемую конфигурацию, необходимую для стабильной и комфортной работы игры “Neon Runner”.

Тип требований	Конфигурация	Комментарий
Минимальные	CPU: Двухъядерный процессор ~2 ГГцОЗУ: 4 ГБГрафика: Встроенная или начального уровня дискретная картаНакопитель: ~500 МБ свободного местаЭкран: 1280x720	Гарантируют запуск с базовой стабильностью при снижении некоторых параметров качества или разрешения
Рекомендуемые	CPU: Четырёхъядерный процессор ~3 ГГцОЗУ: 8 ГБГрафика: Среднего уровня дискретная GPUНакопитель: ~1 ГБ свободного местаЭкран: 1920x1080 или выше	Обеспечивают стабильный 60 FPS, улучшенное качество графики и плавный игровой опыт

При минимальных настройках возможно снижение качества визуальных эффектов и звука для поддержания стабильного фреймрейта. Рекомендуемая конфигурация обеспечивает оптимальное восприятие, позволяя включить все визуальные и звуковые эффекты без заметной потери производительности.

7.2 Программные требования

Программная среда должна соответствовать определённым условиям для гарантии корректной работы игровых механик, рендеринга и звукового сопровождения.

- **Python 3.x:**

Используемая версия Python должна быть актуальной и поддерживаемой (например, Python 3.8 или выше) для обеспечения стабильности и производительности.

- **Pygame последней версии:**

Требуется актуальная версия Pygame (например, Pygame 2.0 или выше), обеспечивающая поддержку современных функций, улучшенную производительность и совместимость с Windows, Linux и macOS.

- **Дополнительные библиотеки (если требуются):**

- Модули для загрузки графики и звуков, поставляемые с Pygame или стандартной библиотекой Python.
- При использовании сторонних библиотек (например, для обработки JSON-файлов с конфигурацией уровней или локализаций) — библиотеки должны быть установлены через менеджер пакетов `pip`.

Все версии зависимостей и библиотек должны быть зафиксированы или задокументированы, чтобы избежать проблем с совместимостью при обновлении компонентов.

8. Сценарии использования (Use Cases)

Ниже представлены ключевые сценарии взаимодействия игрока с игрой, оформленные в формате Use Case. Каждый сценарий определяет последовательность действий и реакцию системы на них, а также акторов и альтернативные потоки выполнения.

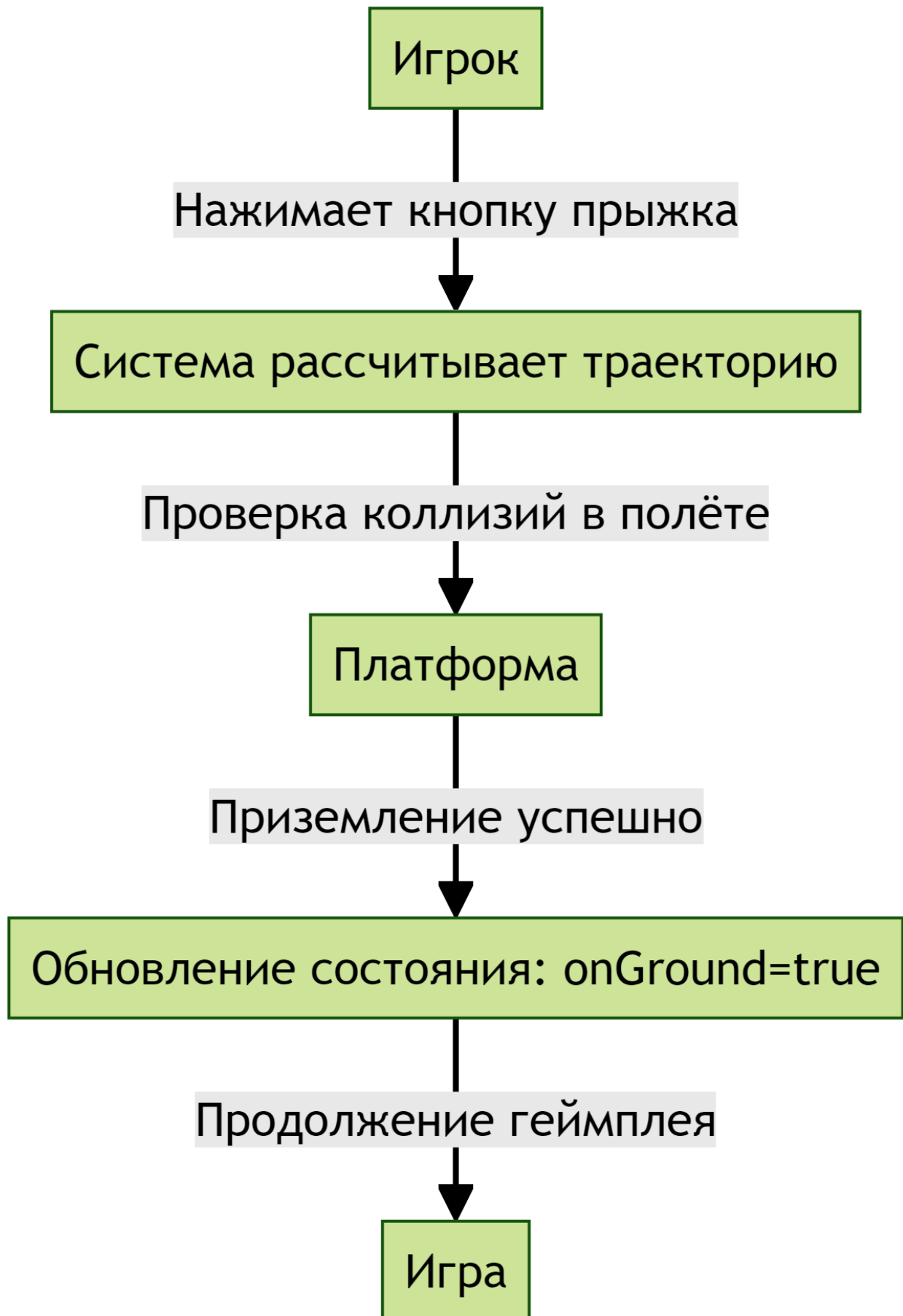
Use Case “Сбор аптечки”



Описание:

Элемент	Детали
Актор	Игрок
Предпосылки	Аптечка размещена на уровне, игрок может к ней добраться
Основной поток	Игрок пересекает хитбокс аптечки → Система добавляет время → Обновляет UI
Альтернативы	Нет (если аптечка труднодоступна, игрок может решить не пытаться её взять)

Use Case “Прыжок на платформу”



Описание:

Элемент	Детали
Актор	Игрок
Предпосылки	Персонаж на твёрдой поверхности или возможно двойной прыжок
Основной поток	Игрок нажимает кнопку → Система обновляет вертикальную скорость → Персонаж приземляется на платформу
Альтернативы	Если нет платформы, игрок падает ниже, теряя время или здоровье

Use Case “Замедление времени для точного прыжка”

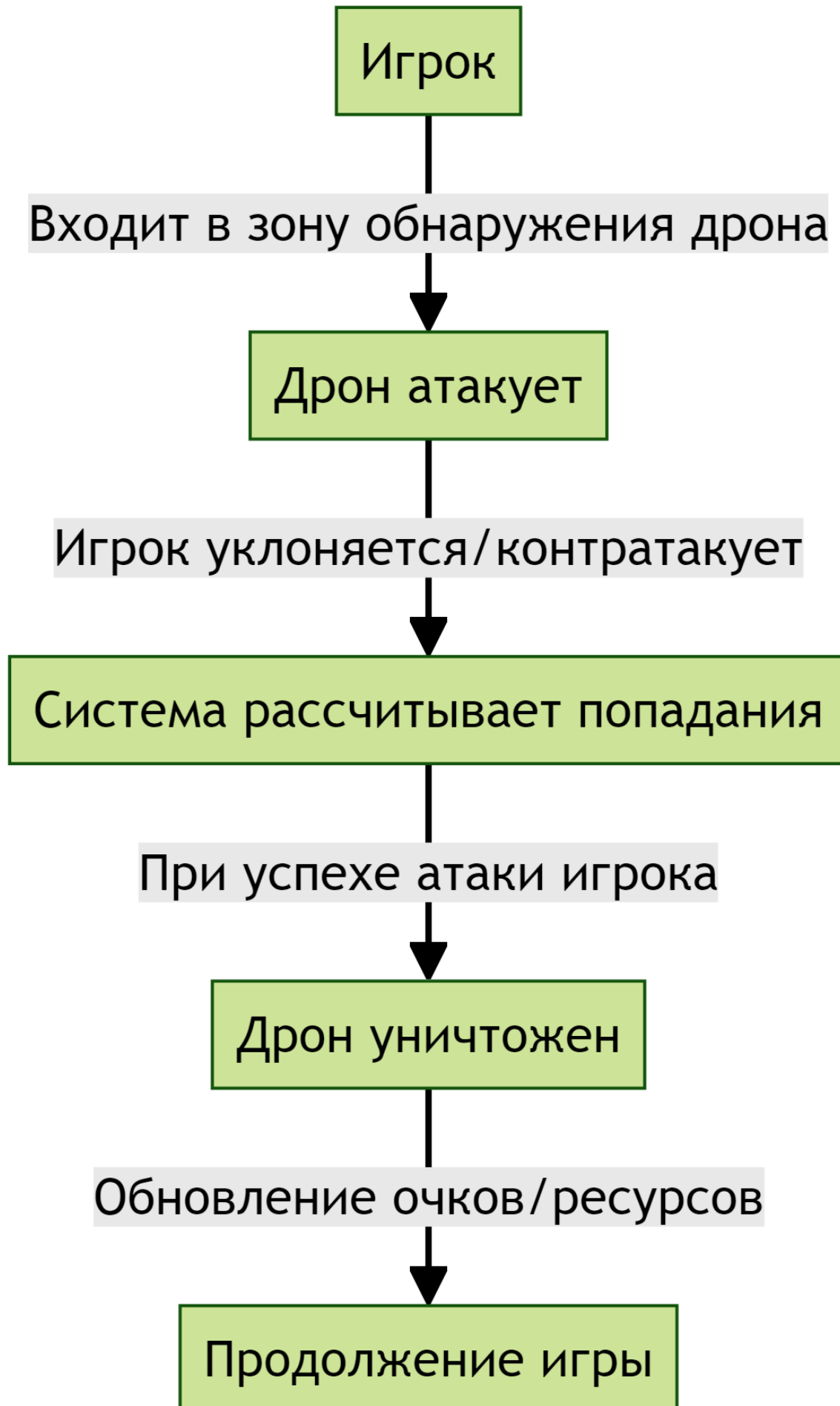


Описание:

Элемент	Детали
Актор	Игрок
Предпосылки	Доступен ресурс замедления времени
Основной поток	Игрок включает замедление → осуществляет прыжок с большим контролем → отключение замедления

Элемент	Детали
Альтернативы	Если ресурс на нуле, замедлить время невозможно

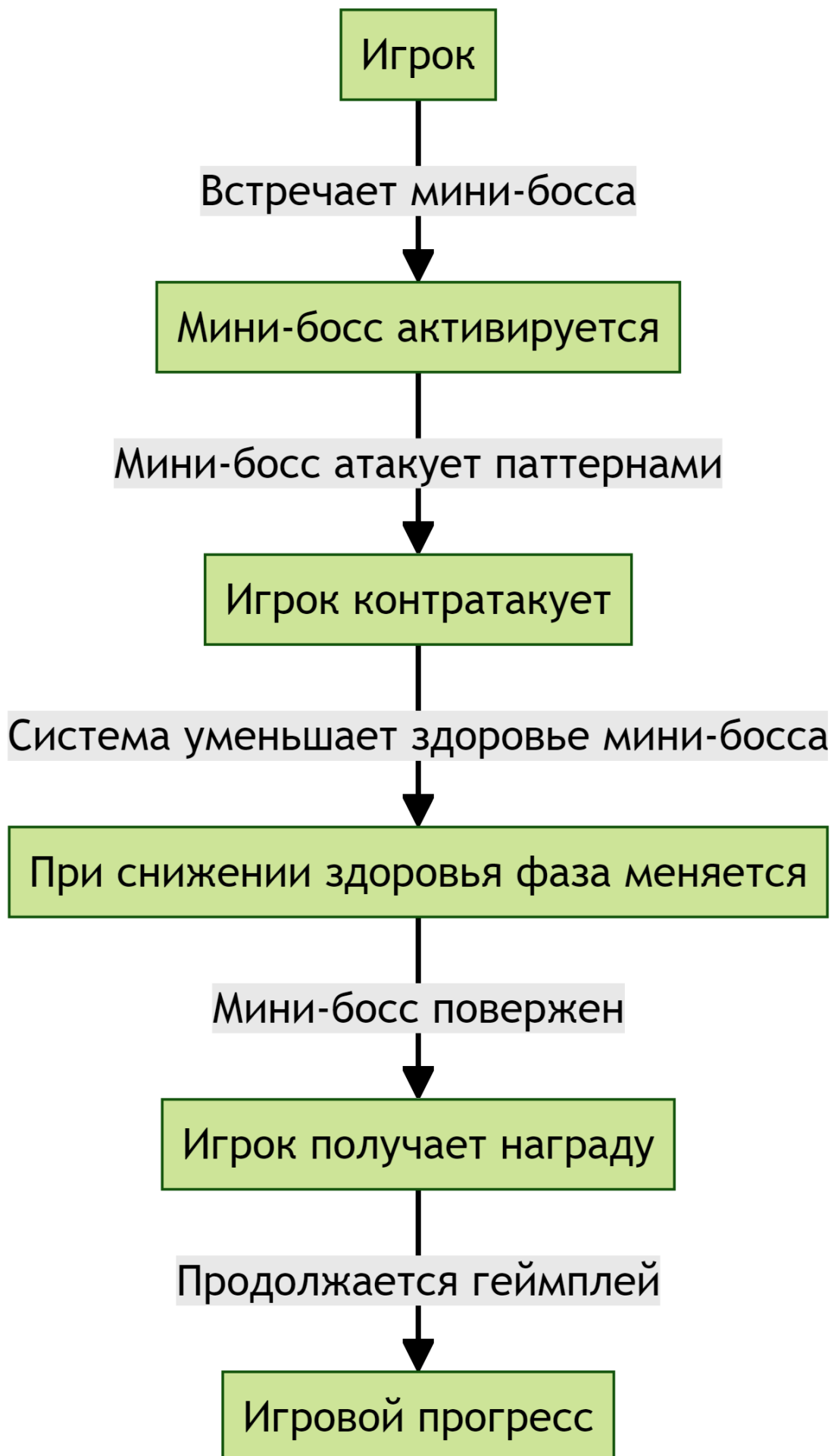
Use Case “Сражение с дроном”



Описание:

Элемент	Детали
Актор	Игрок
Предпосылки	Игрок пересекает радиус агро дрона
Основной поток	Игрок может атаковать (дубинка/пистолет) или уклониться
Альтернативы	Если игрок проигрывает столкновение, теряет здоровье или время

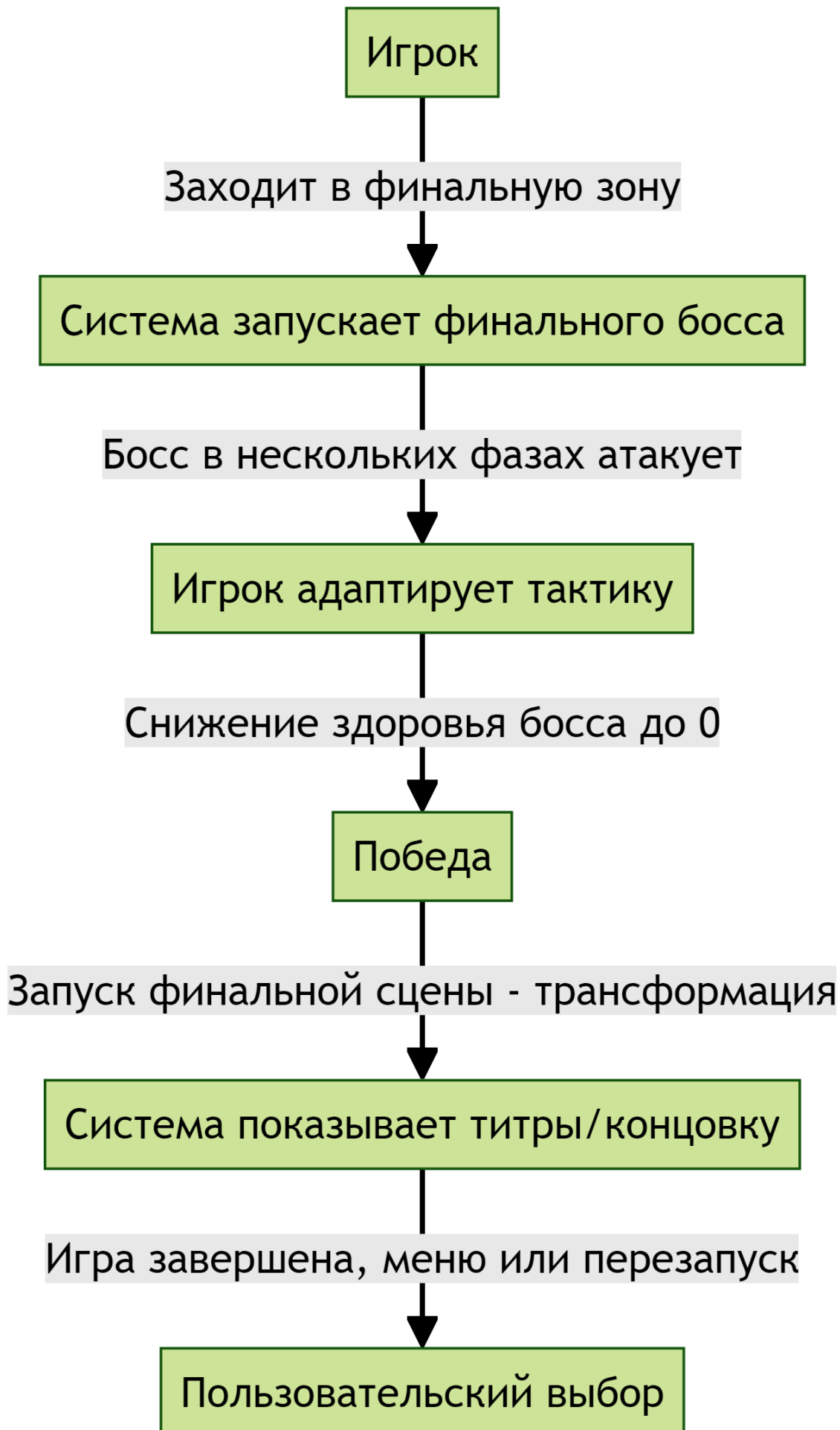
Use Case “Атака мини-босса”



Описание:

Элемент	Детали
Актор	Игрок
Предпосылки	Игрок достиг определённого уровня (этаж)
Основной поток	Игрок сражается, используя оружие и замедление времени
Альтернативы	Если игрок не справляется, может погибнуть или отступить (если возможно)

Use Case “Сцена финального босса и победа”



Описание:

Элемент	Детали
Актор	Игрок
Предпосылки	Игрок прошёл все предыдущие испытания, достиг лаборатории
Основной поток	Сложный бой с финальным боссом, применяя все навыки игрока
Альтернативы	При поражении можно переиграть с последнего чекпоинта

Все эти Use Cases отражают взаимодействие игрока с ключевыми механиками игры “Neon Runner”. Диаграммы наглядно иллюстрируют потоки событий, а таблицы конкретизируют условия, предпосылки и альтернативные пути. Такой подход упрощает понимание и реализацию игровых сценариев в коде.

9. Критерии приемки

- Игра запускается и стабильно работает на целевых платформах.
- Персонаж может свободно двигаться, прыгать, собирать аптечки и использовать замедление времени.
- Все типы врагов корректно реагируют на действия игрока.
- Финальный босс появляется и сражение с ним возможно.
- После победы активируется финальная сцена.
- Жизненный таймер корректно изменяется при сборе аптечек.
- Нет критических ошибок, влияющих на геймплей.

10. Приложения

10.1 Прототипы экранов

- Скетчи интерфейса, уровней, размещение HUD.

10.2 Дополнительные материалы

- Список ассетов (спрайты, звуки).
- Ссылка на исходный GDD.

